Introduction à PyTorch

José Mennesson

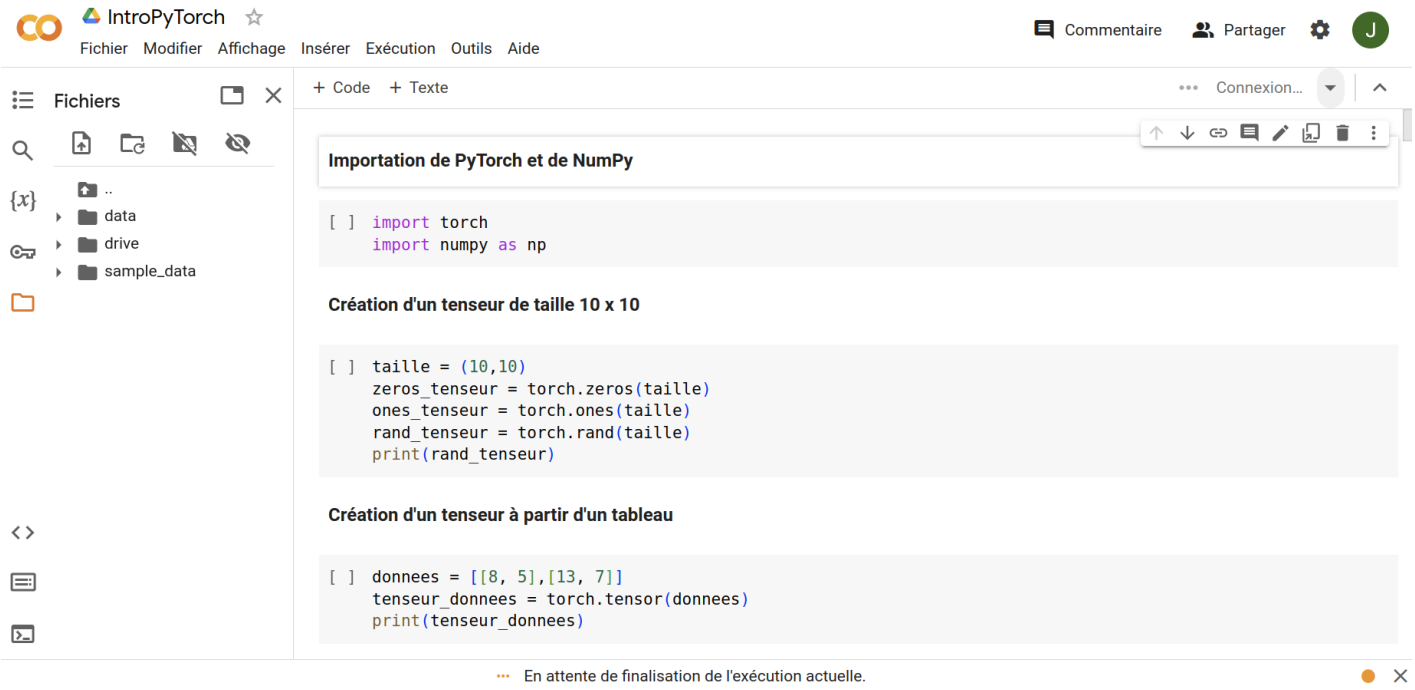
IMT Nord Europe

jose.mennesson@imt-nord-europe.fr (Bureau D107N)

February 12, 2025

Google colab - jupyter-lab

Colab est un service hébergé de bloc-notes Jupyter qui ne nécessite aucune installation et offre un accès gratuit aux ressources informatiques, y compris les GPU et les TPU (Tensor Processing Unit). Colab est particulièrement adapté à l'apprentissage automatique, à la science des données et à l'éducation.

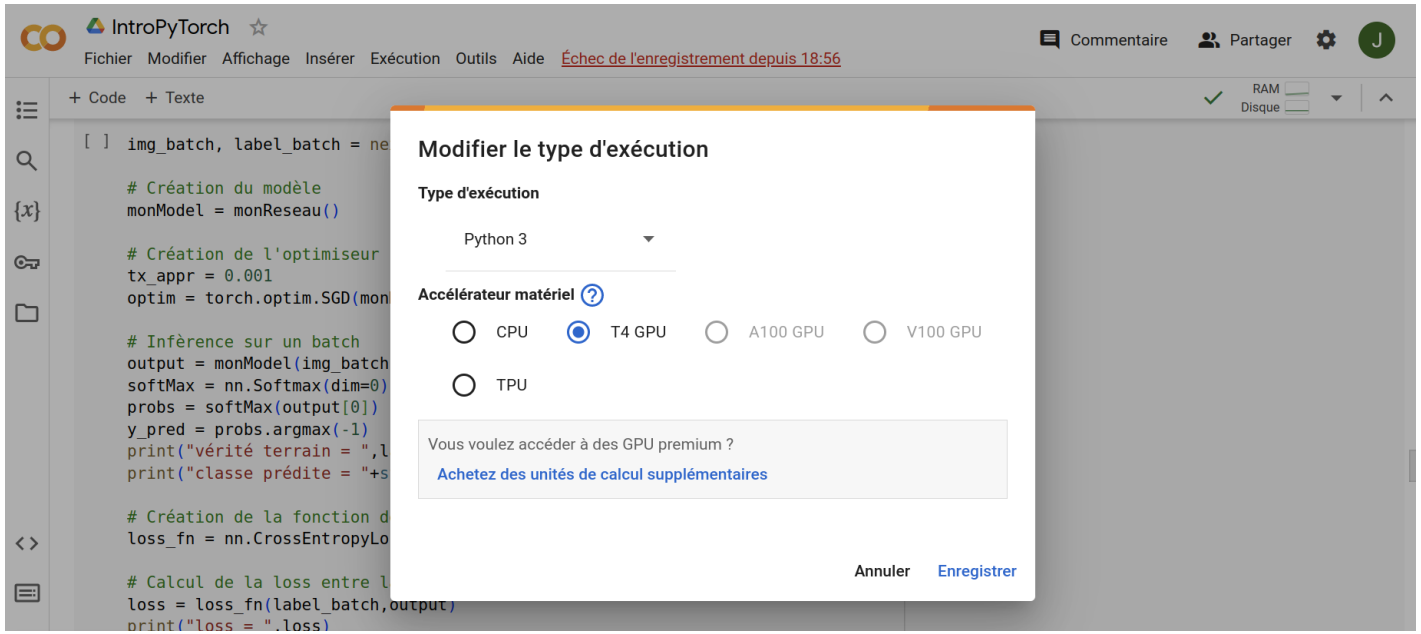


Google colab - jupyter-lab

On peut **Modifier le type d'exécution**.

Google colab - jupyter-lab

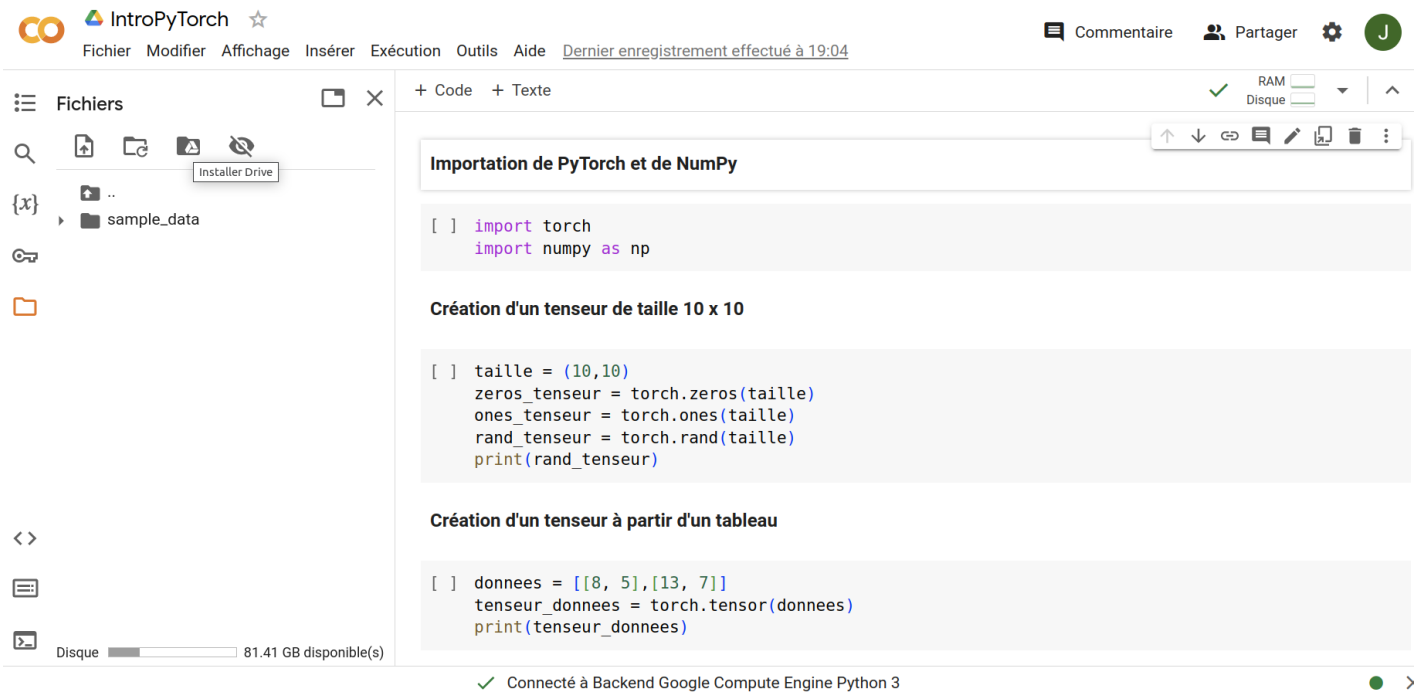
On peut choisir une exécution sur **CPU, GPU et TPU**.



Attention ! Dans la version sans frais de Colab, la durée d'exécution maximale des notebooks est de 12 heures.

Google colab - jupyter-lab

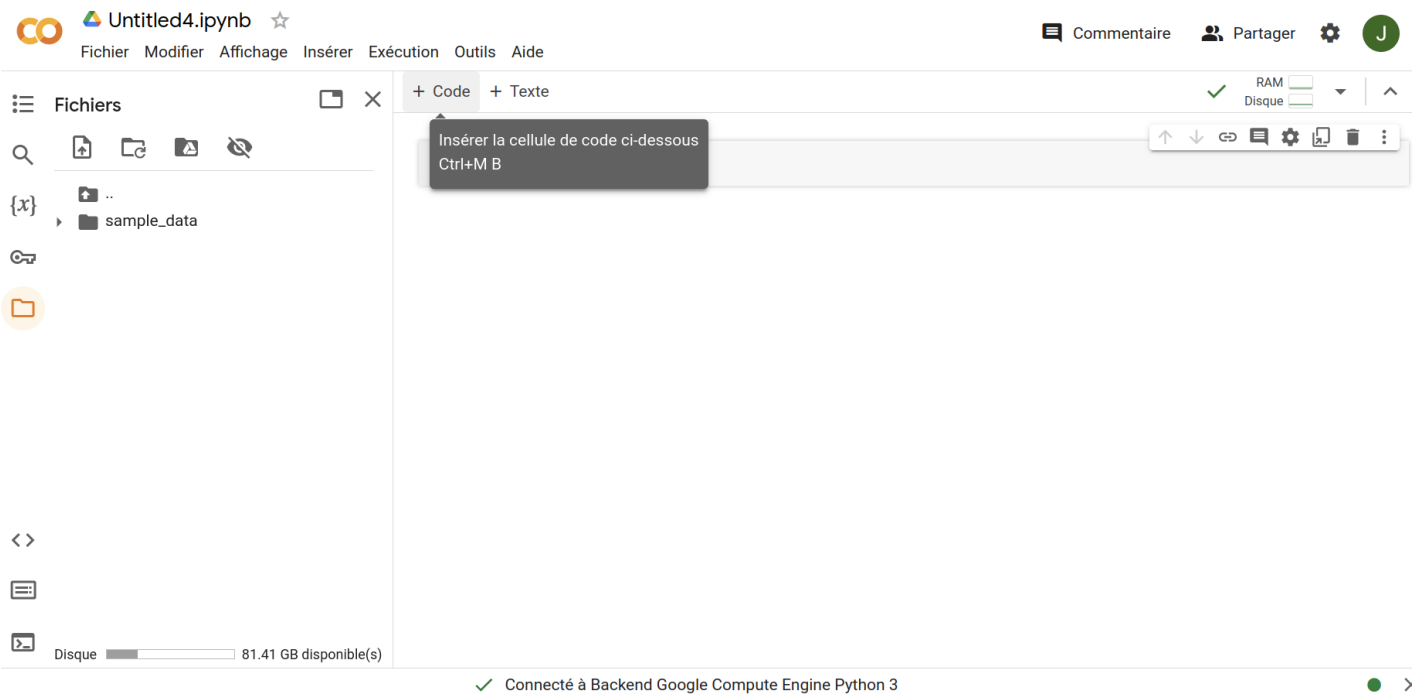
On peut monter son **Google Drive**.



Attention ! Pensez à sauvegarder vos données directement sur votre Drive, sinon, elle seront supprimées automatiquement au bout d'un certain temps.

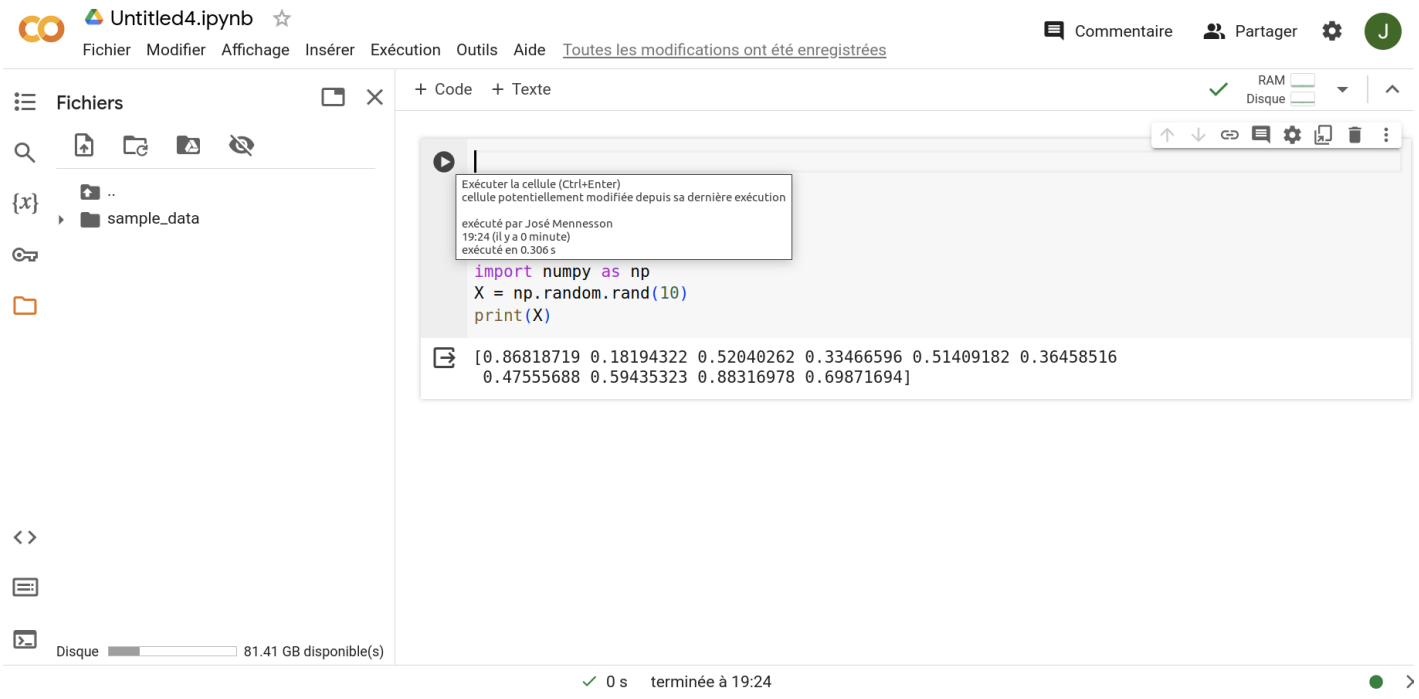
Google colab - jupyter-lab

On peut ajouter des cellules de code Python.



Google colab - jupyter-lab

On peut exécuter une cellule Python.



Pour ajouter une commande système, on préfixe la commande par !. Par exemple :

```
!pip install torchviz
```

Introduction à PyTorch

La bibliothèque PyTorch, qu'est ce que c'est ?

PyTorch :

- est une bibliothèque logicielle Python
- est une librairie open source d'apprentissage machine
- s'appuie sur Torch développée par Meta (Facebook).

PyTorch permet d'effectuer des calculs tensoriels, notamment nécessaires pour l'apprentissage profond (deep learning). Ces calculs sont optimisés et effectués soit par le processeur (CPU) soit, lorsque c'est possible, par un processeur graphique (GPU) supportant CUDA.

Introduction à PyTorch

Installer et importer l'environnement PyTorch

Pour l'installation de PyTorch sur votre machine, rdv sur **https://pytorch.org/get-started/locally/**.

Comme toute bibliothèque Python, on importe la bibliothèque PyTorch tel que :

```
import torch
```

Les tenseurs

Définition

Les **tenseurs** sont

- $\sim$ tableaux ou matrices;
- traitables sur GPU;
- optimisés pour la différenciation automatique.

En PyTorch, on encodera les données d'entrées, de sorties du modèle et les paramètres par des **tenseurs** !

Les tenseurs

Conversion NumPy → tenseur

Tableau NumPy vers tenseur sur **CPU** → la méthode **torch.from_numpy()**.

```
t_np = np.ones(5)
t_cpu = torch.from_numpy(t_np)
```

Attention ! Partage de mémoire entre le tenseur et le tableau NumPy !

Les tenseurs

Conversion tenseur → NumPy

Tenseur sur **CPU** vers un tableau NumPy → la méthode **.numpy()**.
Si tenseur sur **GPU** → **.cpu()** au préalable.

```
t = torch.ones(5)
if torch.cuda.is_available():
  t_cuda = t.to('cuda')
t_cpu = t_cuda.cpu()
t_np = t_cpu.numpy()
t_cpu[3] = 10
t_np[1] = 5
# t_cpu -> tensor([ 1.,  5.,  1., 10.,  1.])
# t_np -> [ 1.  5.  1. 10.  1.]
```

Attention ! Partage de mémoire entre le tenseur et le tableau NumPy !

Les tenseurs

Conversion tenseur 1x1 vers valeur numérique

Il arrive d'avoir besoin de convertir un tenseur de taille 1 (contenant une seule valeur) en un nombre python, pour cela on utilise la fonction **item()**:

```
moy_t = tenseur.sum() # Calcule la moyenne d'un tenseur
# le résultat est un tenseur
moy_v = tenseur_t.item()  # On converti le tenseur en valeur
```

Les tenseurs

Les attributs d'un tenseur

La taille, le type et le device (CPU, GPU) du tenseur sont stockés dans ses attributs **shape, dtype** et **device**.

```
taille = (10,10)
tenseur = torch.rand(taille)
print(tenseur.shape)
print(tenseur.dtype)
print(tenseur.device)
```

Les tenseurs

Les opérations sur les tenseurs

Les opérations sur les tenseurs peuvent être exécutées :

- par défaut sur CPU (beaucoup plus lent que sur GPU car peu de coeurs : 2 $\rightarrow$ 64 grand max.);
- sur GPU en utilisant la librairie CUDA de NVIDIA (plus rapide car exécution parallèle sur des milliers de coeurs CUDA).

Pour utiliser les tenseurs sur GPU, on utilise la méthode **.to(device)** avec device ={'cpu','cuda','mps'}.

```
if torch.cuda.is_available(): # Si CUDA est disponible,
  tensor = tenseur.to('cuda')
  print(tenseur.device)
```

Attention ! Copier de gros tenseurs entre le CPU et le GPU peut consommer beaucoup de temps de calcul et de mémoire !

Les tenseurs

La manipulation de tenseurs comme des tableaux

Les tenseurs sont facilement manipulables comme de simples tableaux. On utilise alors des index pour accéder aux lignes, colonnes, etc.

```
tenseur = torch.rand(4, 4)
print(tenseur[3]) # Affiche la 4ème ligne
print(tenseur[:, 1]) # Affiche la 2ème colonne
print(tenseur[..., -1]) # Affiche la dernière colonne
tenseur[:,1] = 0 # Met 0 dans la 2ème colonne
print(tenseur)
```

Les tenseurs

La concaténation de tenseurs

Pour concaténer des tenseurs selon une dimension, on utilise la fonction **torch.cat**. Pour empiler les tenseurs sur une nouvelle dimension, on utilise plutôt la fonction **torch.stack**.

```
t1 = torch.cat([tenseur, tenseur, tenseur], dim=1)
print(t1)
t2 = torch.stack([tenseur, tenseur, tenseur])
print(t2)
```

Les tenseurs

Les opérations arithmétiques

On utilisera **.T** pour transposer le tenseur.
On peut réaliser des opérations arithmétiques comme :

- des multiplications (opérateur @ ou fonction **matmul**)

```
tenseur = torch.rand(4, 4)
mul1 = tenseur @ tenseur.T
mul2 = tenseur.matmul(tenseur.T)
```

- des multiplications termes à termes (opérateur * ou la fonction **mul**)

```
multat1 = tenseur * tenseur
multat2 = tenseur.mul(tenseur)
```

- des additions.

```
add = tenseur + tenseur
```

- etc.

La gestion des données

Les Datasets et les Dataloaders

Manipulation de bases de données
$\rightarrow$ classes **torch.utils.data.Dataset** et **torch.utils.data.DataLoader**.

- La classe **torch.utils.data.Dataset** est une classe abstraite représentant une base de données. En héritant de cette classe, on peut construire une classe spécialisée pour notre propre base de données. Elle stocke les données et les annotations de ces données.

- La classe **torch.utils.data.DataLoader** permet d'itérer sur la bases de données, créer des **lots (batchs)**, mélanger aléatoirement les données dans les batchs, etc.

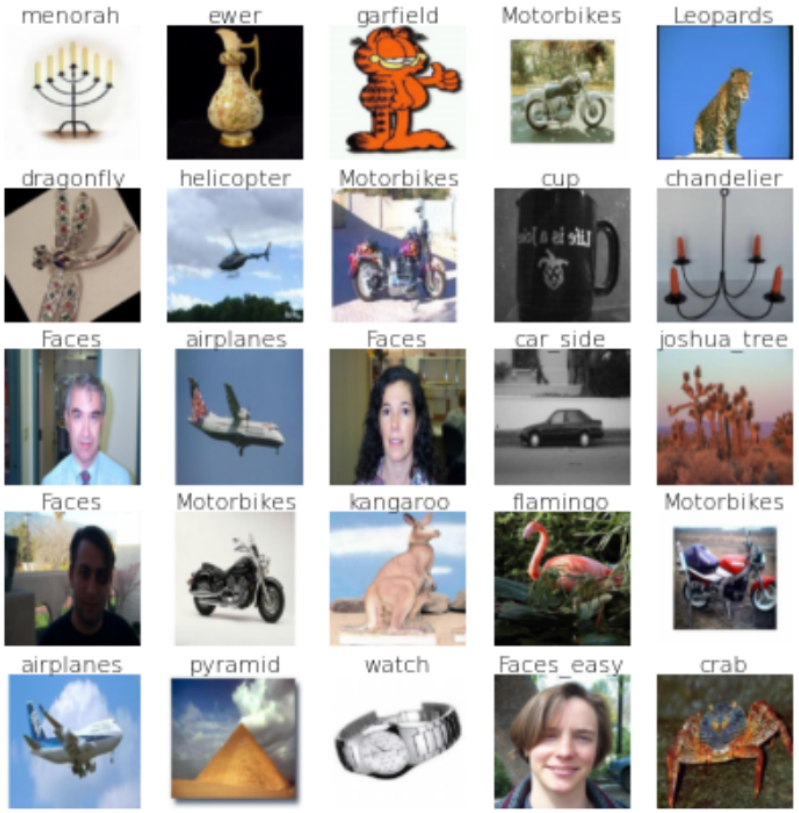
La gestion des données

Les Datasets

PyTorch fournit des sous-classes de **Dataset** dans le package **torchvision** permettant de manipuler facilement des bases d'images et de vidéos connues (voir https://pytorch.org/vision/stable/datasets.html)

Par exemple, la base **Caltech 101**

- ∼9000 images couleur
- de taille ∼ 300 × 200 pixels;
- 101 classes (chien, chat, avion, voiture,...);
- 40 à 800 images par classes;



La gestion des données

La base de données Caltech 101

```
from torchvision import datasets

datasetCaltech101 = datasets.Caltech101(
    root='data',
    target_type = 'category',
    download=True,# on télécharge si pas dans root
)
```

La gestion des données

Affichage des images

Grâce au **dataset**, on peut récupérer les images à l'aide d'un index. On peut utiliser **matplotlib** pour visualiser les images.

```
import matplotlib.pyplot as plt
figure = plt.figure(figsize=(5,5))
cols, rows = 5,5
for i in range(1, cols * rows + 1):
    image_ind = torch.randint(len(datasetCaltech101), size=(1,)).item()
    img, label = datasetCaltech101[image_ind]
    figure.add_subplot(rows, cols, i)
    plt.axis("off")
    plt.imshow(img)
plt.show()
```

La gestion des données

Les transformations

Pour l'apprentissage, on peut appliquer des transformations aux données. Les bases de données pré-chargées de TorchVision ont deux paramètres : **transform** qui modifie les données et **target_transform** qui modifie les annotations.

Ces paramètres acceptent des fonctions de transformation. Le module **torchvision.transforms** proposent différentes transformations couramment utilisées.

La gestion des données

Des exemples de transformation

Pour faire ces transformations, on peut utiliser des fonctions comme :

- **RGB** pour convertir une image niveaux de gris sur 1 canal en image niveaux de gris sur 3 canaux (R,G,B);
- **ToImage** pour convertir une image (H x W x C) ou un tableau NumPy en un **Tensor** (C x H x W);
- **ToDtype** pour mettre à l'échelle les images entre 0 et 1;
- **Resize** qui permet de redimensionner une image;
- **Lambda** qui permet d'appliquer une fonction lambda personnalisée;
- **one_hot** qui permet d'encoder les annotations selon un encodage one_hot.

La gestion des données

La normalisation

La normalisation est une étape importante de pré-traitement commune qui est utilisé pour que chaque donnée contribue équitablement à l'apprentissage.

Cela permet d'éviter :

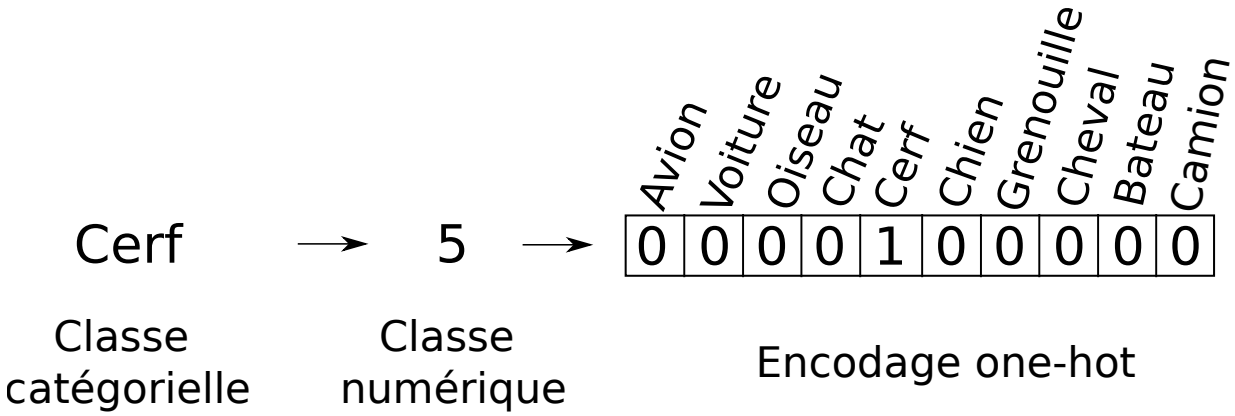
- une réduction de la précision de la prédiction;
- des difficultés de l'apprentissage.

La gestion des données

L'encodage one-hot

Quand on utilise des modèles de classification multiclasses qui produisent des probabilités d'appartenance à chaque classe, on a besoin de représenter les classes différemment.

Le One-hot encoding permet de traiter les étiquettes sous forme vectorielle.



La gestion des données

Application sur Caltech 101

Dans la base Caltech 101, les annotations sont des entiers (classes). Pour l'apprentissage, les données d'entrées doivent être des tenseurs normalisés, sur 3 canaux et de taille égale. Les annotations doivent être des tenseurs de type float encodés selon un encodage **one-hot**.

```
from torchvision.transforms.v2 import Compose, ToImage
from torchvision.transforms.v2 import ToDtype, RGB, Resize
import torch.nn.functional as F

datasetCaltech101 = datasets.Caltech101(
  root="data", download=True,
  transform=Compose([ToImage(), ToDtype(torch.float32, scale=True),
                                                Resize((128,128)),RGB()]),
  target_transform=lambda y:F.one_hot(torch.tensor(y),101).float()
)
```

La gestion des données

Le Dataloader

Utilisation de **lots/batchs** (ensemble d'échantillons) aléatoires pour l'apprentissage.

Objectif : réduire le surapprentissage du modèle et utiliser les capacités multi-tâches de Python pour accélérer la récupération des données.

Un **DataLoader** permet de :

- structurer les données (création de lots);
- les pré-traiter (mélanges, transformations);
- les diffuser aux GPU pour la phase d'entraînement.

La gestion des données

Le Dataloader

Les arguments paramétrables courants du **DataLoader** sont :

- **data** les données utilisées pour le test ou l'entraînement
- **batch size** le nombre d'images à traiter dans chaque lot
- **shuffle** le mélange aléatoire des images
- **pin_memory** permet d'accélérer le transfert des images vers le GPU

```
from torch.utils.data import DataLoader

dataloaderCaltech101 = DataLoader(datasetCaltech101, batch_size=8,
                                  shuffle=True,pin_memory = True)
```

La gestion des données

Le Dataloader

On peut alors itérer sur chacun des lots pour accéder aux images :

```
images, labels = next(iter(dataloaderCaltech101))
print(images.size())
# Récupération d'un batch/tenseur de taille [8, 3, 128, 128]
img = images[0] # Récupère la première image de taille [3, 128, 128]
img = img.swapaxes(0,1) # on swap les dimensions
img = img.swapaxes(1,2) # Récupère une image de taille [128, 128, 3]
plt.imshow(img)
plt.show()
```

Les réseaux de neurones PyTorch

Pour créer des réseaux de neurones sur PyTorch, on utilise le module **nn** de **torch** qui contient tous les outils nécessaires, soit :

```
from torch import nn
```

Chaque module de PyTorch est une sous-classe de nn.Module. Un réseau de neurones est lui-même un module composé d'autres modules (couches). Cette structure imbriquée permet de construire et de gérer facilement des architectures complexes.

Les types classiques de couche

La couche nn.Flatten()

La couche flatten permet de vectoriser (aplatir) les tenseurs données en entrées. Par exemple, une image de taille 32x32 donnée en entrée sera transformée en un vecteur de taille 1024 en sortie.

```
input = torch.rand(1,32,32)
flatten = nn.Flatten()
output = flatten(input)
print(output)
-----------------------------------------------------------
torch.Size([1, 1024])
```

Les types classiques de couche

La couche nn.Linear() (fully-connected)

Chaque caractéristique de l'entrée est connecté à un neurone de la couche linéaire. La couche linéaire applique une transformation linéaire sur l'entrée en utilisant ses poids et son biais (sortie = poids∗entrée+biais).

```
input = torch.rand(1,32*32)
couche_lineaire = nn.Linear(in_features=32*32, out_features=512)
output = couche_lineaire(input)
print(output.size())
-----------------------------------------------------------
torch.Size([1, 512])
```

Les types classiques de couche

La couche nn.Conv2d() (convolution 2D)

La couche nn.Conv2d() réalise des convolutions sur les données en entrées. Elle s'utilise telle que :

```
input = torch.rand(1,32,32)
conv2 = nn.Conv2d(1,6,5) #in_channels, out_channels, kernel_size,
output = conv2(input)
print(output.size())
-----------------------------------------------------------
torch.Size([6, 28, 28])
```

Les types classiques de couche

La couche nn.MaxPool2d() (maxPooling)

La couche nn.MaxPool2d() réalise un max pooling sur les données en entrées. Elle s'utilise telle que :

```
input = torch.rand(1,32,32)
pool = nn.MaxPool2d(2) # kernel_size
output = pool(input)
print(output.size())
-------------------------------------------------------------
torch.Size([1, 16, 16])
```

Il existe d'autres méthodes de Pooling possible : nn.AvgPool2d(), nn.FractionalMaxPool2d(), etc...

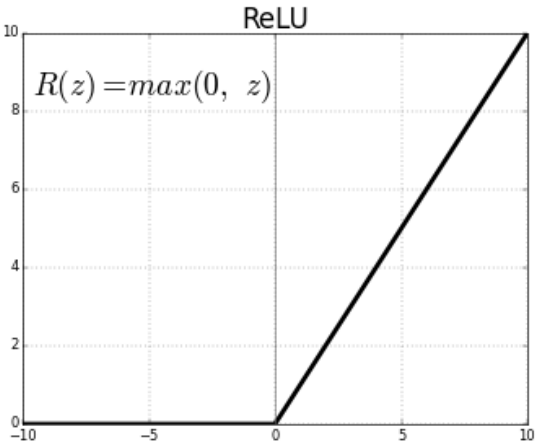
Les types classiques de couche

La fonction d'activation nn.ReLU() (Rectified Linear Unit)

Pour introduire de la non-linéarité dans les modèles, et donc les aider à apprendre des comportements complexes dans les données, on utilise des fonctions d'activation non linéaires.

Une des fonctions d'activation les plus utilisées est la fonction **ReLU** qui prends les valeurs de la sortie de la couche précédente et qui remplace les valeurs négatives par des 0.

```
relu = nn.ReLU()
output = relu(input)
```



Les types classiques de couche

La fonction d'activation nn.Softmax(()

Une couche linéaire d'un réseau de neurones renvoies des valeurs réelles dans $]-\infty\ ;\ +\infty[$.

Si on veut obtenir des probabilités d'appartenance à une classe, on peut utiliser la fonction d'activation **nn.Softmax** qui normalisera les valeur entre 0 et 1 et qui s'arrangera pour que la somme des nombres soit égale à 1.

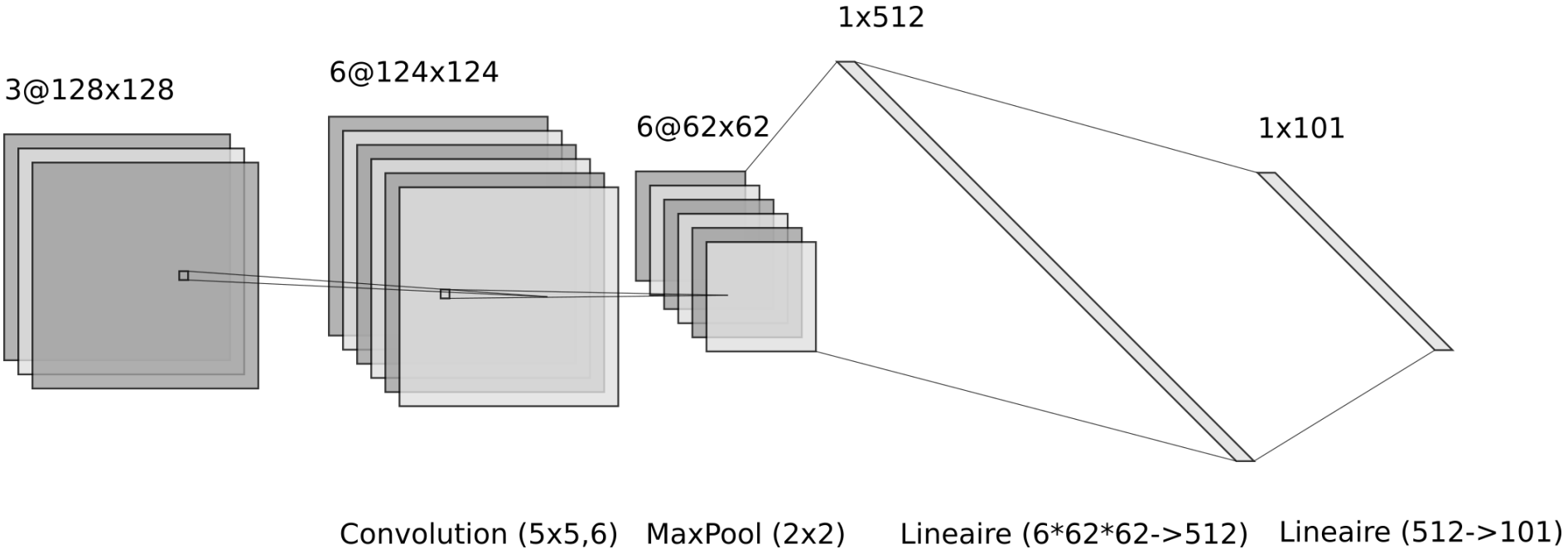
```
input = torch.rand(1,10)
softmax = nn.Softmax(dim=1)
probs = softmax(input)
```

Le paramètre **dim** indique la dimension sur laquelle la somme doit être égale à 1.

Il existe d'autres fonctions d'activations comme **nn.Sigmoid()**, **nn.Tanh()**, **nn.LogSoftmax()**, etc.

La définition d'un réseau complet

Un réseau de neurone convolutionnel



La définition d'un réseau complet

Un réseau de neurone convolutionnel

Pour créer un réseau de neurones, il faut :

1. créer une classe Python qui hérite de **nn.Module**;
2. initialiser chaque couche du réseau de neurones dans la fonction **__init__**;
3. on définit la fonction forward qui précise l'enchainement des couches.

```
class monReseau(nn.Module):
    def __init__(self):
        super(monReseau, self).__init__()        # input  (3,128,128)
        self.conv1 = nn.Conv2d(3, 6, 5)          # output (6,124,124)
        self.pool1 = nn.MaxPool2d(2)             # output (6,62,62)
        self.linear1 = nn.Linear(6*62*62, 512)# linéaire 6*62*62->512
        self.linear2 = nn.Linear(512, 101)   # linéaire 512 -> 101
```

La définition d'un réseau complet

La fonction forward

La fonction forward précise le traitement de l'input (i.e. les images de Caltech101) par les couches initialisée précédemment. Le résultat est alors un vecteur à valeurs dans $]-\infty; 0]$ (après l'application de la fonction **log_softmax()**) .

```
def forward(self, input):
    output = self.pool1(F.relu(self.conv1(input)))
    output = torch.flatten(output,1)
    output = F.relu(self.linear1(output))
    output = F.log_softmax(self.linear2(output),dim=1)
    return output
```

La définition d'un réseau complet

Une instance de réseau

Pour créer un réseau de neurones, on créé une instance de la classe définie auparavant. On peut ensuite afficher sa structure.

```
monModel = monReseau()
print(monModel)
-----------------------------------------------------------------------------
monReseau(
  (conv1): Conv2d(3, 6, kernel_size=(5, 5), stride=(1, 1))
  (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_m
  (linear1): Linear(in_features=23064, out_features=512, bias=True)
  (linear2): Linear(in_features=512, out_features=101, bias=True)
)
```

La définition d'un réseau complet

Utilisation du GPU

Entraîner un réseau de neurones sur un CPU prend un temps considérable. Si on a un GPU de disponible, on passe les paramètres du modèle sur celui-ci à l'aide de la méthode **to** :

```
if torch.cuda.is_available():
      device = 'cuda'
else:
      device = 'cpu'
monModel = monReseau().to(device)
```

La propagation vers l'avant (**forward**)

Dans l'étape de **forward**, le réseau de neurones réalise une prédiction par rapport à la donnée en entrée. Pour ce faire,onl fait passer les données d'entrée dans chacunes des couches/fonctions.

```
img_batch, label_batch = next(iter(dataloaderCaltech101))
output = monModel(img_batch)
y_pred = probs.argmax(-1)
print("vérité terrain = ",label_batch[0].argmax(-1).item())
print("classe prédite = "+str(y_pred[0].item()))
```

Attention ! Le modèle n'est pas entraîné pour le moment ! Les poids et les biais de chaque couche linéaire sont initialisés aléatoirement, et donc le résultat sera donc aléatoire !

Le calcul de l'erreur de prédiction

Pour savoir si la prédiction est juste et/ou à quelle point elle est juste, on va alors calculer une erreur entre la vérité terrain (l'annotation de l'échantillon) et la prédiction réalisée par le réseau de neurones. La fonction qui réalise cela s'appelle la fonction de perte (**loss function**).

Généralement, on utilisera les fonctions de perte suivantes :

- La nn.MSELoss (moyenne des erreurs au carré) pour les tâches de régression;
- La nn.NLLLoss (Negative Log Likelihood) pour les tâches de classification;
- La nn.CrossEntropyLoss qui combine la nn.LogSoftmax et la nn.NLLLoss.

Le calcul de l'erreur de prédiction

Par exemple, on utilise la nn.CrossEntropyLoss pour calculer l'erreur de prédiction :

```
output = monModel(img_batch)
# Création de la fonction de calcul de la loss
loss_fn = nn.CrossEntropyLoss()
# Calcul de la loss entre la vérité terrain et la prédiction
loss = loss_fn(label_batch,output)
print("loss = ",loss)
```

Le calcul des gradients de l'erreur (l'étape de **backward**)

Pour réaliser l'apprentissage du réseau, il faut mettre à jour les paramètres (poids/biais) qui le compose. Pour cela, on utilise la **rétropropagation du gradient de l'erreur**, calculée à l'aide de la fonction de perte.

Pour cela, l'algorithme traverse le réseau de la sortie jusqu'à l'entrée, en collectant les dérivées de l'erreur (les gradients) en fonction des paramètres des fonctions et optimisent les paramètres (poids et biais) en utilisant l'algorithme de descente de gradients. Cette opération est réalisée plusieurs fois avec toutes les données d'apprentissage jusqu'à convergence.

Le calcul des gradients de l'erreur (l'étape de **backward**)

Le package **torch.autograd** est le moteur de différenciation automatique de PyTorch. Il contient des fonctions dédiées comme la fonction **backward** pour calculer automatiquement les gradients. Elle s'utilise tel que :

```
loss.backward()
```

On peut alors visualiser les gradients des paramètres avec :

```
for param in monModel.parameters():
  print(param.grad)
```

Le calcul des gradients

Les paramètres que l'on veut mettre à jour doivent avoir leur attribut **requires_grad** fixé à True. Si ce n'est pas le cas, **autograd** n'en tiendra pas compte dans la rétropropagation du gradient.

De base, les paramètres d'un réseau de neurones ont l'attribut **requires_grad** égal à **True**. Si on veut explicitement créer un tenseur dont on veut calculer le gradient, on fera :

```
tenseur = torch.rand((5, 5), requires_grad=True)
```

Désactiver le calcul des gradients

On peut désactiver le calcul du gradient de certains paramètres lorsque l'on fait du fine tuning par exemple en mettant **requires_grad = False** :

```
for param in model.parameters():
    param.requires_grad = False
```

Dans certains cas, calculer les gradients n'a aucun intéret (pendant la phase de forward par exemple). Pour accélérer les calculs, on peut mettra temporairement les **requires_grad** à **False** en utilisant le bloc **torch.no_grad()** :

```
with torch.no_grad():
        for X, y in dataloader:
            pred = model(X)
```

L'apprentissage du réseau

Pour le moment, nous avons vu comment :

1. gérer les données;
2. créer un réseau de neurones;
3. prédire un résultat en fonction d'une entrée;
4. calculer une erreur de prédiction;
5. calculer les gradients des paramètres du réseau de neurones en fonction de la fonction de perte.

Interressons-nous maintenant à l'apprentissage de notre modèle en utilisant toute notre base de données !

L'apprentissage du réseau

Les hyperparamètres

L'apprentissage va dépendre de plusieurs hyperparamètres qui vont influencer les résultats. On trouve

- Le **nombre d'epochs** qui correspond au nombre de fois que la base de données d'apprentissage entières est présentée au réseau de neurones. Trop peu d'epochs $\rightarrow$ underfitting, trop d'epochs $\rightarrow$ overfitting.
- La **taille de lot (Batch size)** est un hyperparamètre de la descente de gradient qui contrôle le nombre d'échantillons d'apprentissage à traiter avant que les paramètres internes du modèle ne soient mis à jour.
- Le **taux d'apprentissage (learning rate)** est le facteur multiplicatif appliqué au gradient à chaque itération. Il joue sur la rapidité de l'apprentissage. Attention ! Un trop grand learning rate peut provoquer la divergence de l'apprentissage, un trop petit rendra l'apprentissage très long.

L'apprentissage du réseau

L'optimisation des paramètres du réseaux

Pour mettre à jour les paramètres du réseau, une boucle d'optimisation est réalisée. Chaque itération de la boucle correspond à une epoch. Une epoch est divisée en deux parties :

- L'apprentissage qui itère sur la **base de données d'apprentissage** et qui essaye de faire converger les paramètres optimaux.
- La validation/test qui itère sur la **base de données de test** et qui vérifie si les performances du modèle augmentent.

L'apprentissage du réseau

L'optimisation du modèle

L'optimisation correspond à l'étape d'ajustement des paramètres du modèle pour minimiser l'erreur de prédiction à chaque étape de l'apprentissage. On peut alors utiliser des algorithmes comme la descente de gradients.

Différentes méthodes d'optimisation comme

- le Stochastic Gradient Descent (SGD)
- ADAM
- RMSProp

sont disponibles dans la bibliothèque PyTorch.

L'apprentissage du réseau

L'optimisation du modèle

Par exemple, pour le SGD, on l'initialise avec les paramètres du modèle à entrainer et avec le taux d'apprentissage.

```
tx_appr = 0.001
optim = torch.optim.SGD(monModel.parameters(), lr=tx_appr, momentum=0.9)
```

Dans la boucle d'apprentissage, on remet à 0 les gradients des paramètres du modèle, on appelle la fonction **backward()** pour calculer les gradients de l'erreur. Enfin, on fait appel à l'optimiseur.

```
optim.zero_grad()
tx_erreur.backward()
optim.step()
```

L'apprentissage du réseau

model.train() - model.eval()

Par défaut, le modèle est en mode apprentissage. Pour mettre explicitement le modèle en mode apprentissage, on utilisera

```
model.train(True)
```

De la même manière, pour mettre le modèle en mode évaluation,

```
model.eval()
```

L'apprentissage du réseau

La fonction d'apprentissage

```
def train(dataloader, model, loss_fn, optim, device):
    model.train(True)
    nbImages,  sumLoss = 0, 0
    for step, data in enumerate(dataloader):
        inputs, labels = data
        inputs = inputs.to(device)
        labels = labels.to(device)
        pred = model(inputs)
        loss = loss_fn(pred, labels)
        nbImages += labels.size(0)
        sumLoss += loss.item()
        optim.zero_grad()
        loss.backward()
        optim.step()

    print('Images for training')
    images_train, labels_train = next(iter(dataloader))
    showImagesWithLabel(images_train, labels_train, label_names)
    return sumLoss/nbImages
```

L'apprentissage du réseau

La fonction de test/validation

```
def test(dataloader, model, loss_fn, device):
    nbCorrect, nbImages = 0, 0
    model.eval()
    with torch.no_grad(): # On ne calcule pas les gradients
        for data in dataloader:
            inputs, labels = data
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            predicted = outputs.argmax(1)
            labels = labels.argmax(1)
            nbImages += labels.size(0)
            nbCorrect += (predicted == labels).sum().item()

    accuracy = 100 * nbCorrect / nbImages
    print('Accuracy : '+str(accuracy)+' %')
    print('Images for validation')
    showImagesWithLabel(inputs, outputs, label_names)
    return accuracy
```

L'apprentissage du réseau

Itération sur un nombre d'epochs

Il ne reste plus qu'à itérer la boucle d'entrainement et de test sur plusieurs epochs.

```
print("Précision avant apprentissage (poids aléatoires):")
test(dataloaderCaltech101, monModel, loss_fn,device)
for i in range(epochs):
    print('Epoch number : '+str(i+1))
    t.append(i)
    oneLoss = train(dataloaderCaltech101, monModel, loss_fn, optim, device)
    oneAcc = test(dataloaderCaltech101, monModel, loss_fn,device)
    epoch_losses.append(oneLoss)
    epoch_accuracies.append(oneAcc)
    plotAccuracyLoss(t,epoch_accuracies,epoch_losses)
```

L'apprentissage du réseau

L'affichage de la loss et de la précision

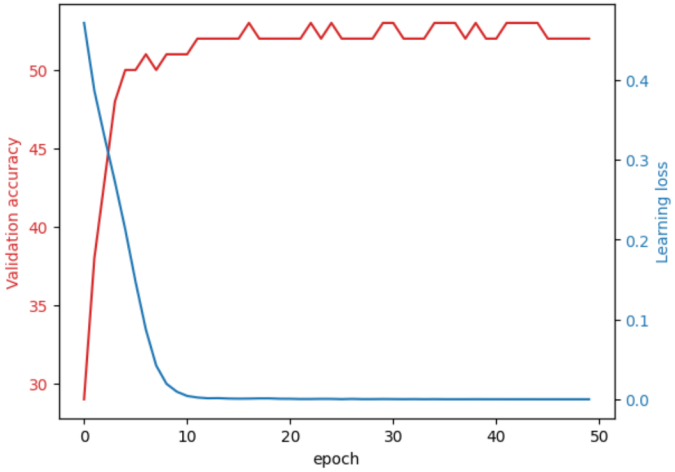
```
def plotAccuracyLoss(t,epoch_accuracies,epoch_losses):
  fig, ax1 = plt.subplots()
  color = 'tab:red'
  ax1.set_xlabel('epoch')
  ax1.set_ylabel('Validation accuracy', color=color)
  ax1.plot(t, epoch_accuracies, color=color)
  ax1.tick_params(axis='y', labelcolor=color)
  ax2 = ax1.twinx()
  color = 'tab:blue'
  ax2.set_ylabel('Learning loss', color=color)
  ax2.plot(t, epoch_losses, color=color)
  ax2.tick_params(axis='y', labelcolor=color)
  fig.tight_layout()
  plt.show()
```

L'apprentissage du réseau

Un exemple

```
Epoch number : 50
Batch : 0 loss : 0.00016824874910525978
Batch : 100 loss : 0.00042138859862461686
Batch : 200 loss : 0.00032789353281259537
Batch : 300 loss : 0.00028578951605595 65
Batch : 400 loss : 0.0015757916262373328
Batch : 500 loss : 0.00048185192281380296
Batch : 600 loss : 0.00029732860275544226
Batch : 700 loss : 0.00035570753971114755
Batch : 800 loss : 0.00020525322179310024
Images for training
```

```
Accuracy : 52 %
Images for validation
```



L'apprentissage du réseau

Le manque de données d'apprentissage

Souvent, on manque de données pour réaliser un apprentissage. Plusieurs solutions :

- les augmentations de données
- le fine tuning

Comment augmenter artificiellement le nombre d'images ?

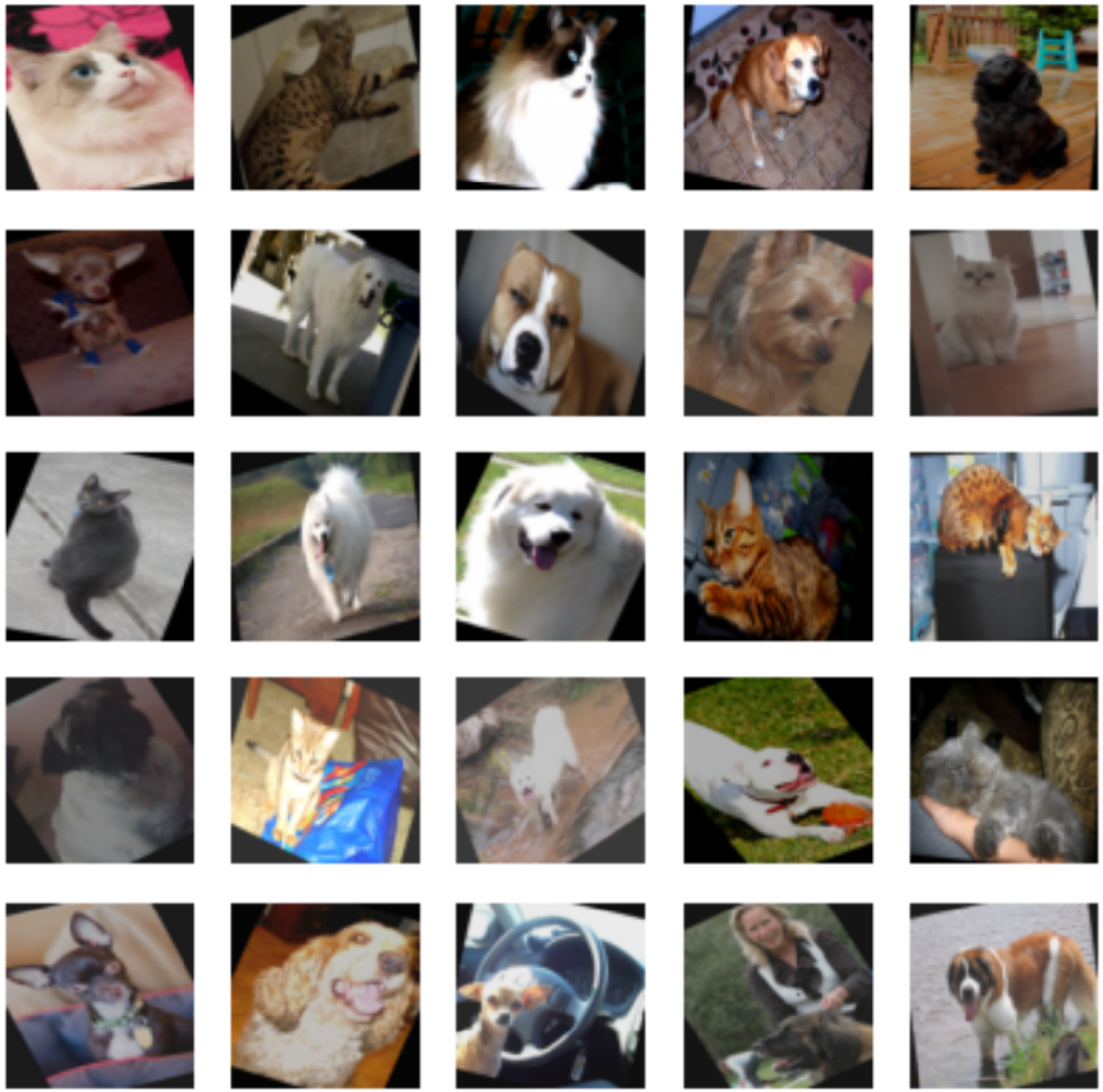
On peut appliquer des transformations simples sur les images pour "fabriquer" de nouvelles données.

On peut agir sur la valeur des pixels (changements aléatoires de luminosité, contraste, saturation, teinte, flou/bruit gaussien, etc.).

On peut agir sur la géométrie de l'image (redimensionnement, changement d'échelle, recadrage, transformation affine, flip horizontal/vertical, etc.).

L'apprentissage du réseau

Exemples d'augmentations



L'apprentissage du réseau

Augmentations : le code Pytorch

```
from torchvision.transforms import v2
train_transform=Compose([ToImage(),
      ToDtype(torch.float32, scale=True),
      RandomRotation(30),
      RandomHorizontalFlip(0.5),
      ColorJitter(brightness=.5,contrast=.5),
      Resize((128,128))
])
```

L'apprentissage du réseau

Le fine tuning

Le **fine tuning** consiste à réaliser un apprentissage en deux phases :

1. apprentissage sur une grande base généraliste (e. g. avec la base imageNet - plus de 10 millions d'images annotées).
2. modification de la dernière couche de classification pour s'adapter au nombre de classes de notre problème (e.g. imageNet = 1000 classes)
3. réalisation d'un apprentissage de **tout le modèle (caractéristiques + classifieur)** sur notre base de données.

L'apprentissage du réseau

Un exemple de fine tuning

```
from torchvision.models import mobilenet_v3_large, MobileNet_V3_Large_Weigh

class myMNV3(nn.Module):
  def __init__(self):
      super(myMNV3, self).__init__()
      self.myMNV3 = mobilenet_v3_large(MobileNet_V3_Large_Weights.DEFAULT)
      num_features = self.myMNV3.classifier[0].in_features
      self.myMNV3.classifier = nn.Linear(num_features, 101)

  def forward(self, input):
      output_enc = self.myMNV3(input)
      output = F.log_softmax(output_enc,dim=1)
      return output
```

L'apprentissage du réseau

La sauvegarde/chargement d'un modèle

On peut sauvegarder les paramètres d'un modèle PyTorch dans un dictionnaire appelé **state_dict** avec la fonction **torch.save** tel que

```
torch.save(monModel.state_dict(), "data/monModel.pth")
```

Pour le charger, on doit créer la classe du réseau de neurone, créer le modèle, puis utiliser la méthode **load_state_dict()** sur le modèle pour charger les paramètres :

```
model = monReseau()
model.load_state_dict(torch.load('data/monModel.pth'))
model.eval() #on met le modèle en mode évaluation !
```